

PRODUCT REQUIREMENTS DOCUMENT

Planora

A secure, JWT-protected event management platform for public and private, free and paid events

Document	Product Requirements Document (PRD)
Version	1.0
Date	4 October 2026
Status	Ready for implementation
Stack	Next.js, Tailwind CSS, Node.js, Express.js, Prisma ORM, PostgreSQL, JWT, SSLCommerz
Scope	Version 1.0 (assignment deliverable)

How to read this document: items marked [ASSUMPTION] are decisions the project brief leaves open. They are listed together in section 1.8. If your instructor or client says otherwise, change them there first, then update the matching business rules (section 8) and logic rules (section 13).

Table of Contents

1. Project Overview	5
1.1 Purpose	5
1.2 Problem and opportunity	5
1.3 Goals	5
1.4 Non-goals for version 1.0	5
1.5 User roles	5
1.6 Event types and join outcome	6
1.7 Delivery requirements and success criteria	6
1.8 Assumptions	6
2. Visual Documentation	7
2.1 System architecture	7
2.2 Site map	7
2.3 Wireframes: Homepage and Event Details	8
2.4 Wireframe: Dashboard	8
2.5 UI conventions	9
3. JWT Authentication Flow	10
3.1 Design decisions	10
3.2 Sequence diagram	10
3.3 Middleware chain on a protected route	10
3.4 Client behavior	11
3.5 Authentication error cases	11
4. Business Logic Flow	12
4.1 Join decision flow	12
4.2 Participation status lifecycle	12
4.3 Paid join, step by step	13
4.4 Invitation flow	13
4.5 Host moderation flow	13
4.6 Review flow	13
4.7 Admin moderation flow	13
5. Core Functionality	14
5.1 Authentication and access	14
5.2 Events	14
5.3 Discovery and homepage	14
5.4 Participation	14
5.5 Invitations	15
5.6 Payments	15

5.7 Reviews and ratings	15
5.8 Dashboard and settings	15
5.9 Admin	15
5.10 Quality requirements	16
6. Product Data Model (PRD Style)	17
6.1 Enumerations	17
6.2 User	17
6.3 Event	17
6.4 Participation	18
6.5 Invitation	18
6.6 Payment	18
6.7 Review	19
6.8 Prisma schema (reference)	19
7. Relationships	21
7.1 Constraints and indexes	21
8. Business Rules	23
8.1 Accounts and access	23
8.2 Events	23
8.3 Participation	23
8.4 Invitations	24
8.5 Payments	24
8.6 Reviews and admin	24
9. API Endpoints	25
9.1 Conventions	25
9.2 Auth and users	25
9.3 Events	25
9.4 Participation	26
9.5 Invitations	26
9.6 Payments	26
9.7 Reviews	26
9.8 Admin	27
9.9 Example: create event	27
10. Project Structure	28
10.1 Server (Express + Prisma)	28
10.2 Client (Next.js App Router)	29
11. Technical Specifications	30
11.1 Environment variables	30

12. Dependencies	31
12.1 Server runtime	31
12.2 Server development	31
12.3 Client runtime	31
12.4 Client development	31
13. Business Logic Rules	32
13.1 BL-01 Join eligibility (used by join, payment init and invitation accept)	32
13.2 BL-02 Join on a free event	32
13.3 BL-03 Payment initiation (POST /payments/init)	32
13.4 BL-04 Payment verification (success callback and IPN)	32
13.5 BL-05 Invitation accept and decline	33
13.6 BL-06 Moderation transitions	33
13.7 BL-07 Reviews	33
13.8 BL-08 Featured event	33
13.9 BL-09 Search and filter mapping	33
13.10 BL-10 Viewer context and field hiding on event details	33
13.11 BL-11 Event update and delete	34
13.12 BL-12 Error handling	34
14. Response Format	35
14.1 Success	35
14.2 Paginated list	35
14.3 Event details with viewer context	35
14.4 Payment initiation	35
14.5 Error	35
14.6 Status codes and error codes	36
15. Deployment Checklist	37
15.1 Before deploying	37
15.2 Database	37
15.3 Server	37
15.4 Client	37
15.5 Payments	37
15.6 Post-deploy smoke test	38
15.7 Submission	38
16. Future Considerations (Post-v1)	39

1. Project Overview

1.1 Purpose

Planora is a secure, JWT-protected web platform where Admins and registered Users can create, manage and participate in events. An event is either Public or Private and either Free or Paid (it may carry a registration fee). This PRD defines the scope, data model, business rules, API, structure and delivery checklist for version 1.0.

1.2 Problem and opportunity

Organizers need one place to publish an event, control who attends and collect registration fees. Attendees need one place to discover events, join free public events instantly, and request or pay to join the rest. Planora combines event publishing, access control, online payment and feedback in a single application.

1.3 Goals

- Let any registered user host events and decide who may attend.
- Support four event types: Public Free, Public Paid, Private Free and Private Paid.
- Collect registration fees online through SSLCommerz and link every payment to a participation request.
- Give hosts approve, reject and ban controls; give Admins platform-wide moderation.
- Let participants rate and review events.
- Deliver a responsive, validated and well-structured Next.js + Express application with clean commit history.

1.4 Non-goals for version 1.0

- Ticket QR codes and on-site check-in.
- Automated refunds and organizer payouts (see assumption A2).
- Real email or SMS delivery. The Notifications setting stores a preference only.
- Event capacity limits and waitlists.
- Image upload for events or profiles.
- Social login and password reset by email.
- A second payment gateway (ShurjoPay is listed under Future Considerations).

1.5 User roles

Role	Description	Key permissions
Guest	Visitor who is not logged in.	Browse home page, events list, event details and reviews. Register and log in.
User	Registered account with role USER.	Everything a Guest can, plus: create, update and delete own events; join, request and pay; accept or decline invitations; write, edit and delete own reviews; manage own settings.
Host (event owner)	The User who created an event. Contextual role, not stored separately.	Approve, reject and ban participants; invite users; edit and delete the event; view participants.
Admin	Account with role ADMIN. Created by a seed script only.	All User abilities, plus: monitor all events and users, delete any event, delete user accounts, choose the featured event.

1.6 Event types and join outcome

Event type	Button on Event Details	Payment	Result for the joiner
Public + Free	Join	No	Joins instantly (status APPROVED).
Public + Paid	Pay & Join	Yes, SSLCommerz	After verified payment: status PENDING until the host approves.
Private + Free	Request to Join	No	Status PENDING until the host approves.
Private + Paid	Pay & Request	Yes, SSLCommerz	After verified payment: status PENDING until the host approves.
Invited user, free event	Accept / Decline	No	Accept gives status APPROVED [ASSUMPTION A1].
Invited user, paid event	Pay & Accept / Decline	Yes, SSLCommerz	After verified payment: status PENDING until the host approves.

1.7 Delivery requirements and success criteria

Requirement	Detail
Commit history	Minimum 20 meaningful commits in the client repository and 20 in the server repository.
Video explanation	5 to 10 minutes. Must show: user registration, login, create event, public free join, paid event payment, private event join request, host approval, dashboard features, admin moderation, event reviews.
Functional	All features in section 5 work end to end for all four event types.
Quality	Validation, loading states and error messages present; layout works on mobile, tablet and desktop; Tailwind styling is consistent; UI components are reusable.

1.8 Assumptions

The brief does not define the points below. These are the decisions this PRD makes so that development has no open questions.

ID	Assumption	Reason
A1	A free invitation that is accepted gives status APPROVED immediately. A paid invitation gives PENDING after payment (as the brief says).	The host already chose to invite this person, so a second approval for a free event adds no value.
A2	Registration fees are not refunded automatically. Refunds, if needed, are handled manually outside the app.	Refunds and payouts are out of scope for v1.
A3	A review can be written only by an APPROVED participant after the event has started. Edit and delete are allowed for 7 days after the review was created.	The brief says delete is allowed 'within review period' without defining it.
A4	For PRIVATE events, venue and event link are shown only to the owner, an Admin and APPROVED participants. Others see 'Visible after approval'.	Protects the location of private events.
A5	REJECTED and BANNED are final for that user and that event: no new request and no new payment.	Prevents repeated charges and spam requests.
A6	SSLCommerz is the single payment gateway. Currency is BDT.	The brief allows 'anything you prefer' and lists SSLCommerz.
A7	The JWT is sent in the Authorization: Bearer header; the client keeps it in a cookie named planora_token.	Works across separately hosted client and server.
A8	Public registration always creates role USER. ADMIN accounts are created by a seed script.	Prevents privilege escalation.
A9	Past events (eventDate in the past) cannot be joined, requested, paid for or invited to.	Avoids pointless registrations and payments.
A10	Only one event is featured at a time, and it must be PUBLIC. If none is featured, the hero shows the soonest upcoming public event.	The hero section needs exactly one event.
A11	Notifications setting stores a boolean preference only. No messages are sent in v1.	Email/SMS delivery is out of scope.

2. Visual Documentation

Diagrams in this document are illustrative blueprints. Wireframes show layout and content, not final colors or typography.

2.1 System architecture

Solid arrow = request | Dashed arrow = response / callback / redirect

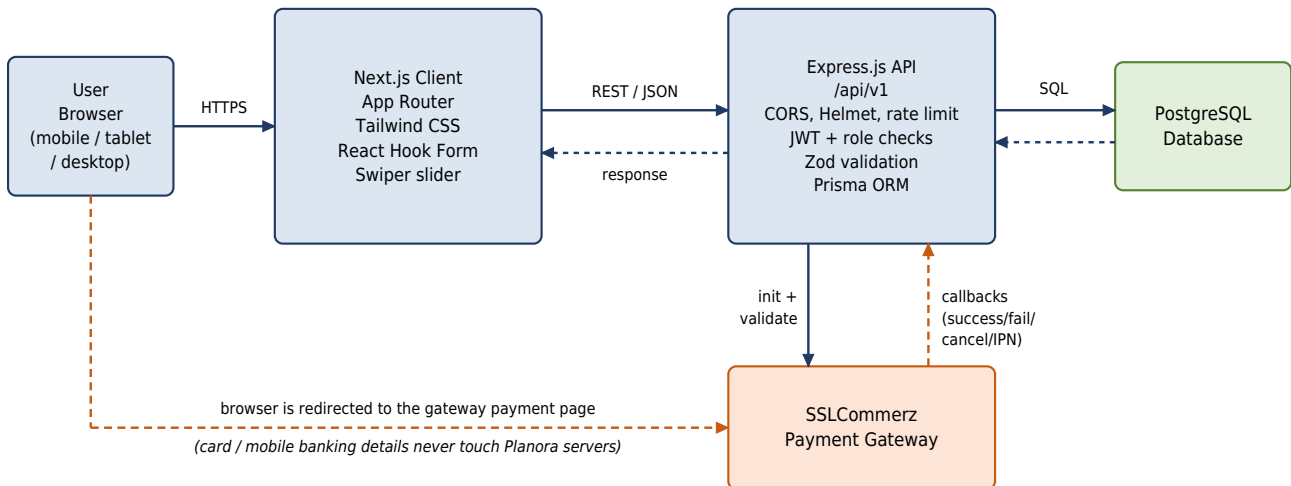
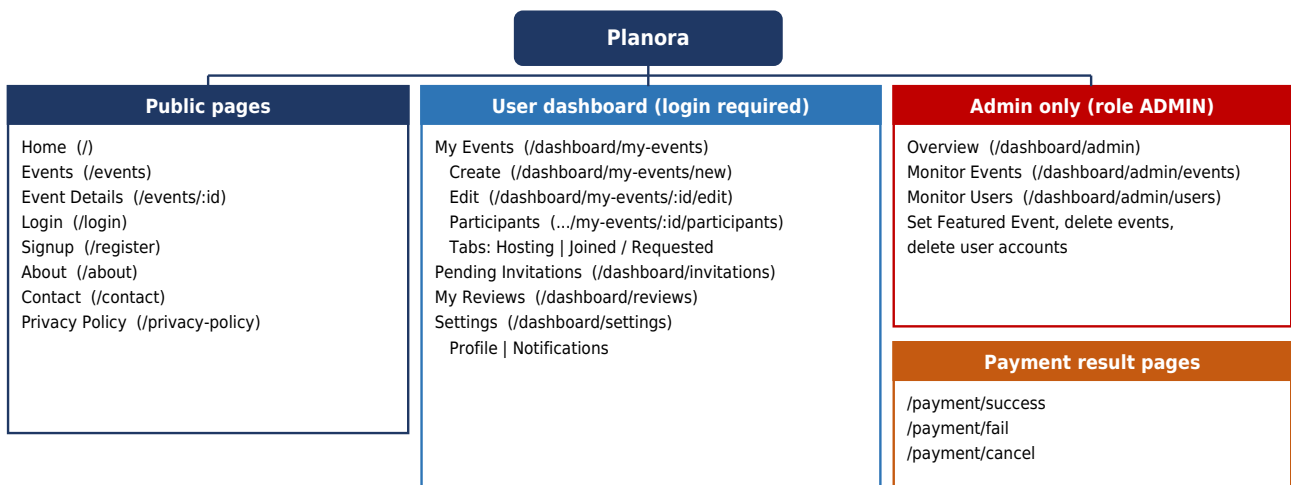


Figure 1. High-level architecture and payment redirect path

- Client and server are two separate repositories so that each has its own commit history (minimum 20 commits each).
- The browser talks only to the Next.js client and, during payment, to the SSLCommerz hosted payment page.
- Only the Express API talks to PostgreSQL and to the SSLCommerz server APIs. The store password never reaches the browser.

2.2 Site map



Navbar: Home | Events | Login / Signup | Dashboard. Footer: About | Contact | Privacy Policy.

Guests can open public pages only. /dashboard/* needs a valid JWT; /dashboard/admin/* also needs role ADMIN.

Figure 2. Pages and routes

2.3 Wireframes: Homepage and Event Details

Homepage

[Planora](#) [Home](#) [Events](#) [Login / Signup](#) [Dashboard](#)

HERO: Featured event (selected by Admin)

Event title | Event date
Short description

[Join](#)

Upcoming Events Slider (9 public events) < >

Event title
Date | Organizer
[Free] / [Paid]

Event title
Date | Organizer
[Free] / [Paid]

Event title
Date | Organizer
[Free] / [Paid]

Event Categories (filters)

[Public Free](#) [Public Paid](#) [Private Free](#) [Private Paid](#)

CALL TO ACTION

[Create events](#) | [Join events](#)

Footer: [About](#) | [Contact](#) | [Privacy Policy](#)

Event Details page

[Planora](#) [Home](#) [Events](#) [Login / Signup](#) [Dashboard](#)

Event Title [Public](#) [Paid](#)

Date and Time

Venue or Event Link

Organizer

Registration Fee

Description

ACTION: [Join](#) | [Pay & Join](#) | [Request to Join](#) | [Pay & Request](#)
(one button, chosen by event type)

Owner controls (event owner only)

[Approve](#) [Reject](#) [Ban](#) [Edit](#) [Delete](#)

Reviews and Ratings

★★★★☆ 4.0 - user review text ...

[Write review] [Edit] [Delete] (within review period)

Figure 3. Homepage (left) and Event Details page (right)

2.4 Wireframe: Dashboard

[Planora](#) [Home](#) [Events](#) [user menu]

[My Events](#)

[Pending Invitations](#)

[My Reviews](#)

[Settings](#)

My Events

Tabs: [Hosting](#) | [Joined / Requested](#)

Title	Date	Type	Fee	Participants	Actions
Tech Meetup	12 Nov 2026	Public	Free	24	Edit Delete Manage
Design Workshop	20 Nov 2026	Private	500 BDT	8	Edit Delete Manage
Startup Pitch Night	05 Dec 2026	Public	200 BDT	41	Edit Delete Manage

Manage opens the participants view: pending requests with Approve / Reject, approved participants with Ban.

[+ Create Event](#)

Figure 4. Dashboard with sidebar (My Events view)

2.5 UI conventions

Element	Specification
Breakpoints	Tailwind defaults: mobile (below 640 px), tablet (md, 768 px and up), desktop (lg, 1024 px and up). Sidebar collapses to a drawer below lg.
Fee badge	Free = green badge. Paid = amber badge showing the amount, for example 'Paid 500 BDT'.
Visibility badge	Public = blue. Private = purple.
Status badges	PENDING = yellow, APPROVED = green, REJECTED = red, BANNED = dark gray. Invitation: PENDING yellow, ACCEPTED green, DECLINED gray. Payment: SUCCESS green, FAILED red, CANCELLED gray, PENDING yellow.
Reusable components	Button, Input, Textarea, Select, Card, EventCard, Badge, Modal, ConfirmDialog, Spinner, Skeleton, Pagination, StarRating, EmptyState, Toast, FormField, DataTable, Sidebar, Navbar, Footer.
Loading and errors	Skeleton for lists, spinner on buttons during submit, full-page 'Processing payment...' state on payment result pages, toast for success and error messages, inline messages under invalid fields.
Accessibility	Labels on all inputs, visible focus ring, sufficient color contrast, buttons are real button elements, images and icons have text alternatives.

3. JWT Authentication Flow

3.1 Design decisions

Topic	Decision
Token type	JWT signed with HS256 using the secret JWT_SECRET (random, at least 32 characters, never committed).
Payload	sub = user id, role = USER or ADMIN, iat, exp. No personal data beyond id and role.
Lifetime	7 days (JWT_EXPIRES_IN=7d). No refresh token in v1.
Transport	Request header Authorization: Bearer followed by the token.
Client storage	Cookie planora_token (SameSite=Lax, Secure on HTTPS, 7 days). A cookie lets Next.js middleware gate /dashboard routes. This is UI gating only; the API enforces real security.
Password hashing	bcrypt (package bcryptjs), cost factor 10. Plain passwords are never stored or logged.
Password rules	Minimum 8 characters, at least one letter and one number.
Logout	Stateless: the client deletes the cookie. Tokens are not revocable in v1, but a token for a deleted account is rejected because the user is loaded from the database on every request.
Role enforcement	Server-side middleware: authenticate, optionalAuthenticate and requireRole('ADMIN'). Owner checks live in the service layer.

3.2 Sequence diagram

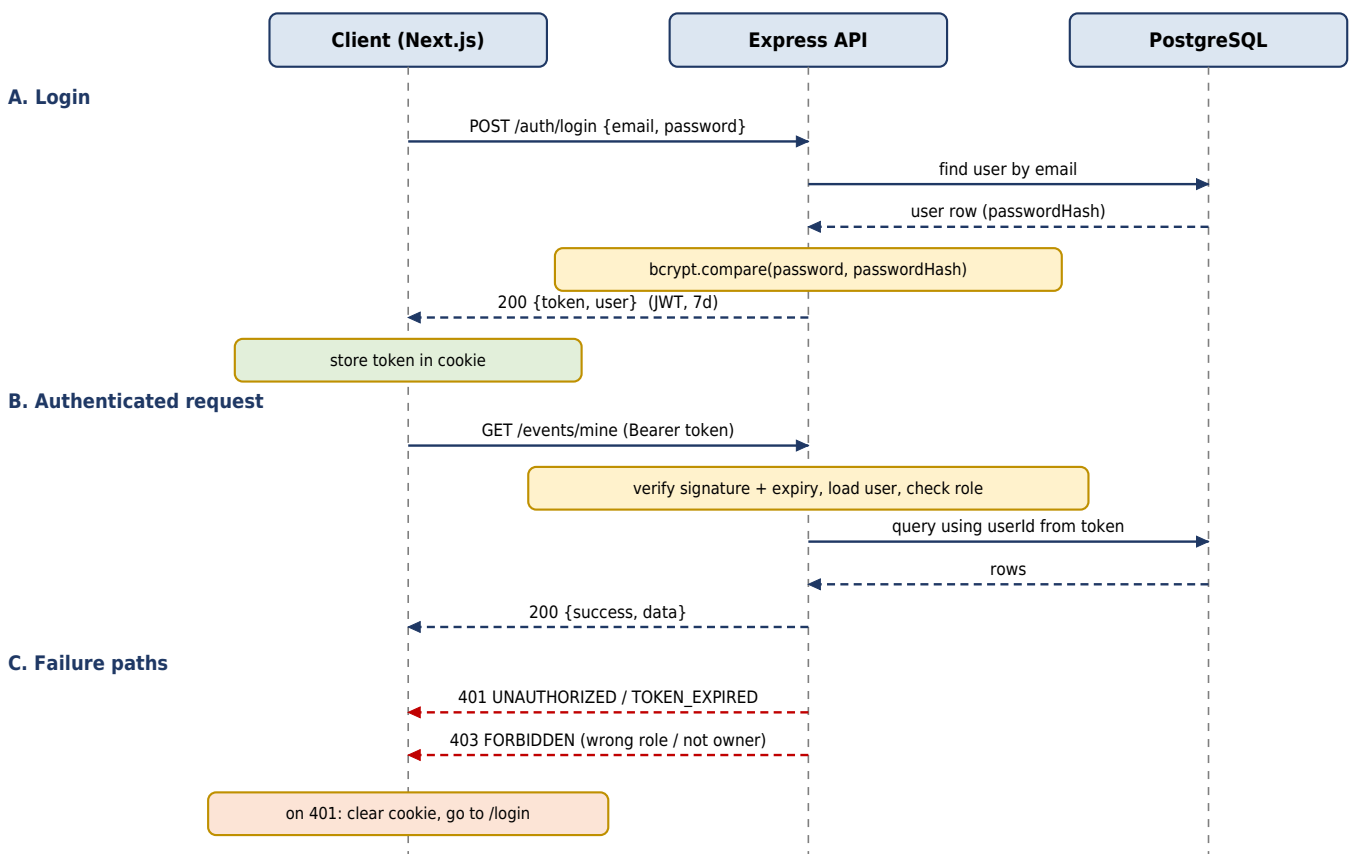


Figure 5. Login, authenticated request and failure paths

3.3 Middleware chain on a protected route

- helmet, cors (origin = CLIENT_URL), express.json, express.urlencoded (needed for gateway callbacks), rate limiter.
- authenticate: read the Bearer token, verify signature and expiry, load the user by sub, attach `req.user = { id, role }`. Any failure returns 401.
- requireRole('ADMIN') on /admin routes. Wrong role returns 403.

- validate(zodSchema): parse body, query and params. Failure returns 400 with field errors.
- controller, then service (business rules, Prisma), then sendResponse. Errors flow to the central error handler.

3.4 Client behavior

- After login or register: save token in the cookie and user in the auth context, then redirect to the dashboard or the page the user came from.
- An axios request interceptor adds the Authorization header when the cookie exists.
- An axios response interceptor handles 401 by clearing the cookie and redirecting to /login with a message.
- Next.js middleware redirects visitors without the cookie from /dashboard/* to /login. Admin pages also check the role from /auth/me on the client and the server rejects non-admins with 403.
- Navbar shows Login / Signup for guests and Dashboard plus a user menu for logged-in users.

3.5 Authentication error cases

Situation	HTTP	errorCode	Message shown
Wrong email or password	401	UNAUTHORIZED	Invalid email or password (same message for both, so accounts cannot be probed).
No token on a protected route	401	UNAUTHORIZED	Authentication required.
Malformed or tampered token	401	UNAUTHORIZED	Invalid token.
Expired token	401	TOKEN_EXPIRED	Session expired. Please log in again.
Token for a deleted user	401	UNAUTHORIZED	Account no longer exists.
Valid token, insufficient role or not the owner	403	FORBIDDEN	You do not have permission to do this.
Email already registered	409	CONFLICT	An account with this email already exists.
Too many login attempts	429	RATE_LIMITED	Too many attempts. Try again later.

4. Business Logic Flow

This section shows how the main flows behave from the user's point of view. Exact algorithms and error codes are in section 13.

4.1 Join decision flow

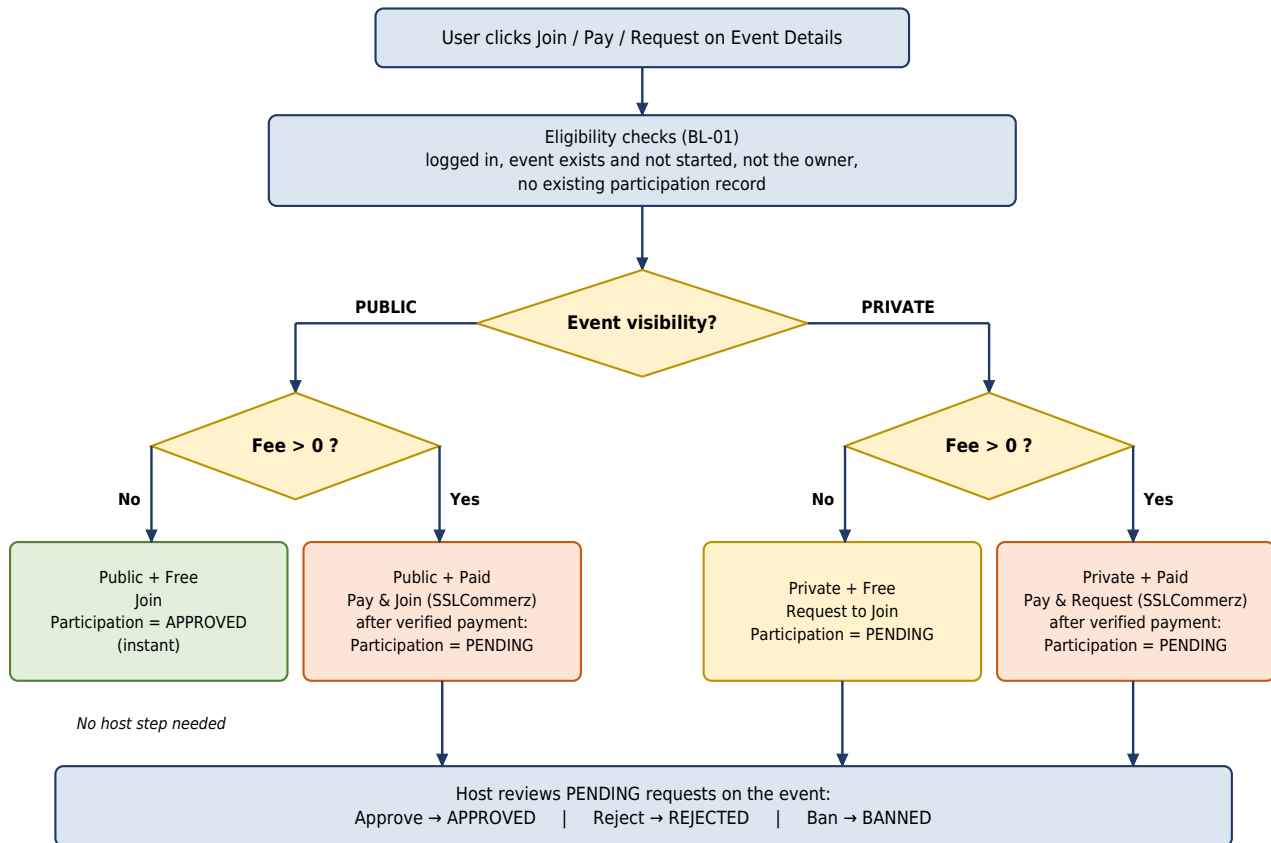


Figure 6. What happens when a user joins, requests or pays

4.2 Participation status lifecycle

REJECTED and BANNED are final for that user and event (no re-request, no payment).

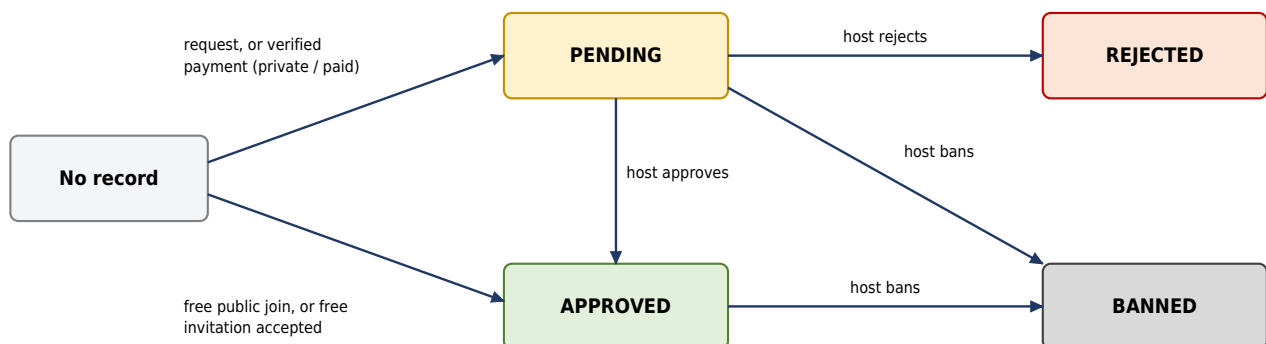


Figure 7. Participation status transitions

4.3 Paid join, step by step

Step	Actor	Action
1	User	Clicks Pay & Join (or Pay & Request, or Pay & Accept) on a paid event.
2	Client	Calls POST /payments/init with eventId (and invitationId when paying an invitation).
3	Server	Checks eligibility, creates a Payment row with status PENDING and a unique tranId, calls the SSLCommerz session API, returns gatewayUrl.
4	Client	Redirects the browser to gatewayUrl.
5	User	Pays on the SSLCommerz hosted page.
6	SSLCommerz	Sends the browser (POST) to the server success, fail or cancel URL, and sends a server-to-server IPN call.
7	Server	On success: validates the payment with the gateway validation API (val_id), checks amount, currency and tranId, then in one transaction marks Payment SUCCESS and creates Participation with status PENDING (and marks the invitation ACCEPTED if one was paid).
8	Server	Redirects the browser to CLIENT_URL/payment/success (or /payment/fail, /payment/cancel) with tran_id.
9	Client	Payment result page shows the outcome and links back to the event. Failure shows 'Payment failed' with a retry button.
10	Host	Sees the new PENDING request and approves or rejects it.

4.4 Invitation flow

- Host opens the event participants view and invites a registered user by email.
- Invitee sees the invitation in Dashboard, Pending Invitations, with event title, date, organizer and fee.
- Free event: Accept creates Participation with status APPROVED; Decline marks the invitation DECLINED.
- Paid event: the Accept button reads Pay & Accept and starts the payment flow in 4.3. After verified payment the status is PENDING (host approval still needed).
- Decline is allowed for both free and paid invitations and needs no payment.

4.5 Host moderation flow

- Host opens Dashboard, My Events, Manage (participants view) and filters by status.
- PENDING row: Approve moves it to APPROVED; Reject moves it to REJECTED.
- PENDING or APPROVED row: Ban moves it to BANNED.
- Host may edit the event (limits in BR-09) or delete it. Deleting removes participations, invitations and reviews but keeps payment records.

4.6 Review flow

- After the event has started, an APPROVED participant sees a review form on the Event Details page (rating 1 to 5, optional comment).
- One review per user per event. The author can edit or delete it for 7 days after creation; after that it is read-only.
- All visitors see the average rating, the review count and the list of reviews. The author also sees their reviews in Dashboard, My Reviews.

4.7 Admin moderation flow

- Admin opens Dashboard, Admin and sees counts for users, events, participations and successful payments.
- Monitor Events: search and list all events; delete an inappropriate event; set or clear the Featured event.
- Monitor Users: list all users; delete a user account (not an Admin account, not themselves).

5. Core Functionality

Priority: M = must have for v1, S = should have. Acceptance criteria describe the observable result.

5.1 Authentication and access

ID	Feature	Description and acceptance criteria	Role
F-01	Register	Form with name, email, password. Email format and password rules validated. Creates a USER, returns token and user, redirects to dashboard. Duplicate email shows an error.	Guest
F-02	Login	Email and password. Success stores token and redirects. Wrong credentials show 'Invalid email or password'.	Guest
F-03	Session and logout	GET /auth/me restores the user after refresh. Logout clears the cookie and returns to home.	User
F-04	Role-based access	Admin routes return 403 for non-admins. Dashboard routes redirect guests to /login. Unauthorized attempts show a clear message.	All

5.2 Events

ID	Feature	Description and acceptance criteria	Role
F-05	Create event	Fields: title, date, time, venue, event link (venue or link required), description, Public/Private, registration fee (0 = free). All validated. The new event appears in My Events.	User
F-06	Update event	Owner edits fields. Visibility and fee are locked once the event has participations, invitations or payments (BR-09).	Owner
F-07	Delete event	Owner or Admin deletes after a confirmation dialog. Related participations, invitations and reviews are removed; payments are kept.	Owner, Admin
F-08	Event details	Shows title, date and time, venue or link (see A4), description, organizer, registration fee, rating summary and the correct action button for the viewer.	All

5.3 Discovery and homepage

ID	Feature	Description and acceptance criteria	Role
F-09	Hero section	Featured event chosen by Admin: title, date, short description, Join button (goes to Event Details). Fallback per A10.	All
F-10	Upcoming slider	Slider with up to 9 upcoming PUBLIC events. Each card: title, date, organizer, Free/Paid badge.	All
F-11	Event categories	Four filters (Public Free, Public Paid, Private Free, Private Paid). Each links to /events with that filter applied.	All
F-12	Call to action	Section encouraging users to create and join events; links to /register or the create form.	All
F-13	Events page	Search by title or organizer, the four filters, pagination. Card shows title, date, organizer, fee, View Details button.	All
F-14	Layout	Navbar: Home, Events, Login / Signup, Dashboard. Footer: About, Contact, Privacy Policy.	All

5.4 Participation

ID	Feature	Description and acceptance criteria	Role
F-15	Join, request, pay	Button depends on type: Join, Pay & Join, Request to Join, Pay & Request. Outcome per section 1.6. Button state reflects the viewer's current status (Pending, Joined, Rejected, Banned).	User
F-16	Participant management	Owner sees participants with status, filters by status, and can Approve, Reject or Ban.	Owner
F-17	My participation	My Events has a 'Joined / Requested' tab listing the user's participations with status.	User

5.5 Invitations

ID	Feature	Description and acceptance criteria	Role
F-18	Invite user	Owner enters the email of a registered user. Errors for unknown email, duplicate invitation, existing participation or inviting self.	Owner
F-19	Respond to invitation	Invitee sees Accept / Decline (free) or Pay & Accept / Decline (paid) in Pending Invitations.	User

5.6 Payments

ID	Feature	Description and acceptance criteria	Role
F-20	Start payment	POST /payments/init creates a pending payment and returns the SSLCommerz page URL. Loading state is shown during the call.	User
F-21	Verify payment	Success, fail, cancel and IPN endpoints update the payment and create the participation only after server-side validation (BL-04). Repeated callbacks do not duplicate anything.	System
F-22	Payment result pages	/payment/success, /payment/fail, /payment/cancel show the outcome and a link back to the event. Failure shows 'Payment failed'.	User

5.7 Reviews and ratings

ID	Feature	Description and acceptance criteria	Role
F-23	Write review	Rating 1 to 5, optional comment. Only for APPROVED participants after the event starts; one per event.	User
F-24	Edit and delete review	Author only, within 7 days of creation. After that the buttons are hidden and the API returns 403.	User
F-25	My Reviews	Dashboard page listing the user's reviews with edit and delete actions (where still allowed).	User

5.8 Dashboard and settings

ID	Feature	Description and acceptance criteria	Role
F-26	Dashboard layout	Sidebar: My Events, Pending Invitations, My Reviews, Settings (plus Admin links for Admins). Responsive drawer on small screens.	User
F-27	Settings	Update profile (name, phone) and Notifications preference (A11).	User

5.9 Admin

ID	Feature	Description and acceptance criteria	Role
F-28	Monitor	Overview counts; searchable lists of all events and all users.	Admin
F-29	Delete content	Delete inappropriate events and user accounts (BR-04, BR-05, BR-34).	Admin
F-30	Featured event	Set or clear the featured event shown in the hero section.	Admin

5.10 Quality requirements

ID	Feature	Description and acceptance criteria	Role
F-31	Validation	Required fields, email format, fee (number, 0 or more, 2 decimals). Same rules on client (Zod) and server (Zod).	All
F-32	Loading states	API loading (skeletons, button spinners) and payment processing.	All
F-33	Error messages	Invalid login, payment failure, unauthorized access, not found, network error, each with a clear message.	All
F-34	Responsive UI	Usable on mobile, tablet and desktop with consistent Tailwind styling and a clean layout.	All
F-35	Reusable components	Shared UI components as listed in section 2.5.	Dev

6. Product Data Model (PRD Style)

Six entities. Field types use Prisma names. A trailing ? means optional. PK = primary key, FK = foreign key, UQ = unique.

6.1 Enumerations

Enum	Values	Meaning
Role	ADMIN, USER	Account role.
Visibility	PUBLIC, PRIVATE	Public: join directly if free. Private: join by request.
ParticipationStatus	PENDING, APPROVED, REJECTED, BANNED	State of a user's participation in an event.
InvitationStatus	PENDING, ACCEPTED, DECLINED	State of an invitation.
PaymentStatus	PENDING, SUCCESS, FAILED, CANCELLED	State of a gateway payment.

6.2 User

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
name	String	required, 2-80 chars	Display name; used as the organizer name.
email	String	UQ, stored lowercase	Login identifier.
passwordHash	String	required	bcrypt hash. Never returned by the API.
role	Role	default USER	ADMIN or USER.
phone	String?	optional	Profile field; also sent to the gateway.
notificationsEnabled	Boolean	default true	Notification preference.
createdAt, updatedAt	DateTime	auto	Timestamps.

6.3 Event

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
title	String	required, 3-120 chars	Event title.
description	String	required, 10-5000 chars	Full description. The hero shows the first 160 characters as the short description.
eventDate	DateTime	required, future on create	Date and time together (UTC). The form collects Date and Time separately and combines them.
venue	String?	optional	Physical place.
eventLink	String?	optional, valid URL	Online link. At least one of venue or eventLink is required.
visibility	Visibility	required	PUBLIC or PRIVATE.
fee	Decimal(10,2)	default 0, 0 to 1,000,000	Registration fee in BDT. 0 = Free, above 0 = Paid.
isFeatured	Boolean	default false	True for the hero event (one at a time, PUBLIC only).
organizerId	String	FK to User, cascade	Owner and host.
createdAt, updatedAt	DateTime	auto	Timestamps.

6.4 Participation

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
eventId	String	FK to Event, cascade	Event joined.
userId	String	FK to User, cascade	Participant.
status	ParticipationStatus	required	Current status.
decidedAt	DateTime?	optional	When the host last approved, rejected or banned.
createdAt, updatedAt	DateTime	auto	Timestamps.
(eventId, userId)	-	UQ	One participation per user per event.

6.5 Invitation

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
eventId	String	FK to Event, cascade	Event the user is invited to.
inviteId	String	FK to User, cascade	Invited user.
status	InvitationStatus	default PENDING	Current status.
respondedAt	DateTime?	optional	When accepted or declined.
createdAt	DateTime	auto	Timestamp.
(eventId, inviteId)	-	UQ	One invitation per user per event.

6.6 Payment

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
tranId	String	UQ	Transaction id sent to the gateway, for example PLN-xxxxxxx.
userId	String?	FK to User, SetNull	Payer. Null if the account is deleted.
eventId	String?	FK to Event, SetNull	Event paid for. Null if the event is deleted.
eventTitle	String	required	Snapshot of the title for records.
amount	Decimal(10,2)	required	Copied from event.fee by the server.
currency	String	default BDT	Currency.
status	PaymentStatus	default PENDING	Gateway outcome.
gateway	String	default SSLCOMMERZ	Gateway name.
valId, bankTranId	String?	optional	Gateway validation id and bank transaction id.
participationId	String?	FK, SetNull	Participation created by this payment.
invitationId	String?	FK, SetNull	Invitation paid, if any.
createdAt, paidAt	DateTime, DateTime?	auto, optional	Created and confirmed times.

6.7 Review

Field	Type	Constraints	Description
id	String (uuid)	PK	Identifier.
eventId	String	FK to Event, cascade	Reviewed event.
userId	String	FK to User, cascade	Author.
rating	Int	1 to 5	Star rating.
comment	String?	optional, up to 1000 chars	Review text.
createdAt, updatedAt	DateTime	auto	createdAt starts the 7-day edit window.
(eventId, userId)	-	UQ	One review per user per event.

6.8 Prisma schema (reference)

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role          { ADMIN USER }
enum Visibility    { PUBLIC PRIVATE }
enum ParticipationStatus { PENDING APPROVED REJECTED BANNED }
enum InvitationStatus { PENDING ACCEPTED DECLINED }
enum PaymentStatus { PENDING SUCCESS FAILED CANCELLED }

model User {
  id          String      @id @default(uuid())
  name        String
  email       String      @unique
  passwordHash String
  role        Role        @default(USER)
  phone       String?
  notificationsEnabled Boolean @default(true)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  events      Event[]
  participations Participation[]
  invitations  Invitation[]
  payments     Payment[]
  reviews     Review[]
}

model Event {
  id          String      @id @default(uuid())
  title       String
  description  String
  eventDate   DateTime
  venue       String?
  eventLink   String?
  visibility  Visibility
  fee         Decimal      @default(0) @db.Decimal(10, 2)
  isFeatured  Boolean      @default(false)
  organizerId String
  organizer   User         @relation(fields: [organizerId], references: [id], onDelete: Cascade)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  participations Participation[]
  invitations  Invitation[]
  payments     Payment[]
  reviews     Review[]

  @@index([visibility, eventDate])
  @@index([organizerId])
}

model Participation {
  id          String      @id @default(uuid())
  eventId     String
  userId      String
  status      ParticipationStatus
  decidedAt   DateTime?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  event       Event        @relation(fields: [eventId], references: [id], onDelete: Cascade)
  user        User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  payments     Payment[]

  @@unique([eventId, userId])
}

```

```

    @@index([eventId, status])
}

model Invitation {
  id          String          @id @default(uuid())
  eventId     String
  inviteeId   String
  status      InvitationStatus @default(PENDING)
  respondedAt DateTime?
  createdAt   DateTime        @default(now())
  event       Event           @relation(fields: [eventId], references: [id], onDelete: Cascade)
  invitee     User            @relation(fields: [inviteeId], references: [id], onDelete: Cascade)
  payments    Payment[]

  @@unique([eventId, inviteeId])
  @@index([inviteeId, status])
}

model Payment {
  id          String          @id @default(uuid())
  tranId      String          @unique
  userId      String?
  eventId     String?
  eventTitle   String
  amount      Decimal         @db.Decimal(10, 2)
  currency    String          @default("BDT")
  status      PaymentStatus   @default(PENDING)
  gateway     String          @default("SSLCOMMERZ")
  valId       String?
  bankTranId  String?
  participationId String?
  invitationId String?
  createdAt   DateTime        @default(now())
  paidAt      DateTime?
  user        User?           @relation(fields: [userId], references: [id], onDelete: SetNull)
  event       Event?          @relation(fields: [eventId], references: [id], onDelete: SetNull)
  participation Participation? @relation(fields: [participationId], references: [id], onDelete: SetNull)
  invitation   Invitation?     @relation(fields: [invitationId], references: [id], onDelete: SetNull)

  @@index([userId])
  @@index([eventId])
}

model Review {
  id          String          @id @default(uuid())
  eventId     String
  userId      String
  rating      Int
  comment     String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  event       Event       @relation(fields: [eventId], references: [id], onDelete: Cascade)
  user        User        @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([eventId, userId])
}

```

7. Relationships

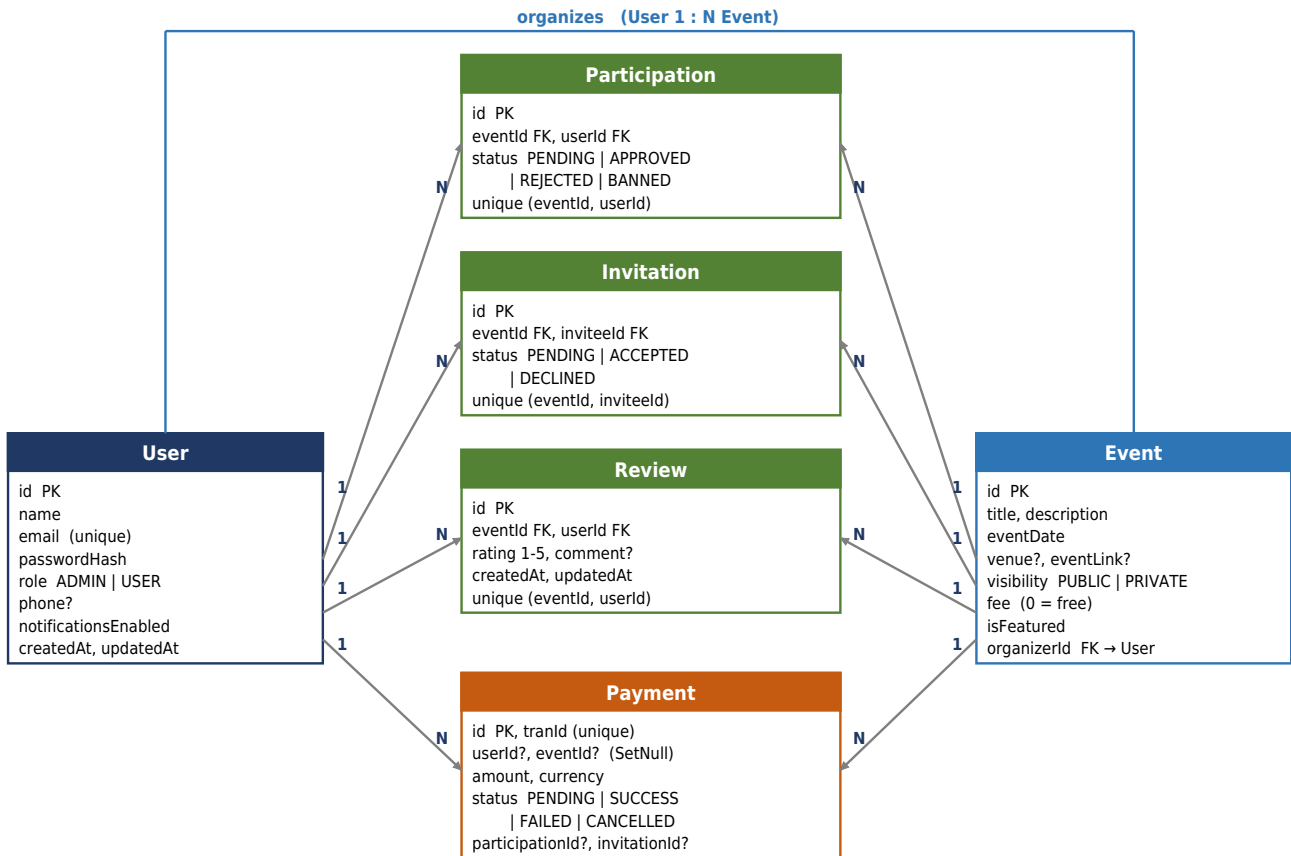


Figure 8. Entity relationship diagram (1 = one, N = many)

From	Relationship	To	Foreign key	On delete
User	1 : N organizes	Event	Event.organizerId	Cascade: deleting a user deletes their events
User	1 : N takes part in	Participation	Participation.userId	Cascade
Event	1 : N has	Participation	Participation.eventId	Cascade
User	1 : N receives	Invitation	Invitation.inviteId	Cascade
Event	1 : N has	Invitation	Invitation.eventId	Cascade
User	1 : N writes	Review	Review.userId	Cascade
Event	1 : N receives	Review	Review.eventId	Cascade
User	1 : N makes	Payment	Payment.userId (optional)	SetNull: payment kept
Event	1 : N is paid for by	Payment	Payment.eventId (optional)	SetNull: payment kept
Participation	1 : N created by	Payment	Payment.participationId (optional)	SetNull
Invitation	1 : N paid by	Payment	Payment.invitationId (optional)	SetNull

7.1 Constraints and indexes

- Unique: User.email, Payment.tranId, Participation(eventId, userId), Invitation(eventId, inviteId), Review(eventId, userId).
- Indexes: Event(visibility, eventDate) for listing and the slider; Event(organizerId) for My Events; Participation(eventId, status) for host views; Invitation(inviteId, status) for Pending Invitations; Payment(userId), Payment(eventId).

- User and Event are the two aggregate roots. Participation, Invitation, Review and Payment always belong to a user and an event (Payment keeps history even if they are deleted).
- A Payment links to a Participation only after the payment succeeds, so Payment.participationId is empty while the payment is pending.

8. Business Rules

Rules describe what must always be true. The procedures that enforce them are in section 13.

8.1 Accounts and access

ID	Rule
BR-01	Email is unique and compared in lowercase.
BR-02	Public registration always creates role USER. ADMIN accounts exist only through the seed script.
BR-03	Guests may view home, events list, event details and reviews. Every other action requires login.
BR-04	An Admin has all User abilities. An Admin cannot delete their own account or another Admin account.
BR-05	Deleting a user deletes their events, participations, invitations and reviews. Their payment records are kept with userId set to null.

8.2 Events

ID	Rule
BR-06	Required on create: title, description, eventDate (in the future), visibility, fee, and at least one of venue or eventLink.
BR-07	fee is a number from 0 to 1,000,000 with at most 2 decimals. fee = 0 means Free; fee above 0 means Paid.
BR-08	Only the owner can update an event. The owner or an Admin can delete it.
BR-09	Once an event has any participation, invitation or payment, its visibility and fee cannot change. Other fields remain editable. eventDate cannot be moved to the past.
BR-10	An event whose eventDate is in the past cannot be joined, requested, paid for or invited to (A9).
BR-11	Only an Admin can feature an event. Only a PUBLIC event can be featured. At most one event is featured at a time (A10).
BR-12	The homepage slider shows up to 9 PUBLIC events with eventDate in the future, soonest first.
BR-13	Venue and eventLink of a PRIVATE event are visible only to the owner, Admins and APPROVED participants (A4).
BR-14	Organizer name shown on cards is the owner's User.name.

8.3 Participation

ID	Rule
BR-15	A user has at most one participation per event.
BR-16	The owner cannot join, request, pay for or be invited to their own event.
BR-17	Initial status: Public + Free = APPROVED. Public + Paid = PENDING after verified payment. Private + Free = PENDING. Private + Paid = PENDING after verified payment.
BR-18	For paid events a participation is created only after the payment is verified. No payment, no participation.
BR-19	REJECTED and BANNED users cannot request again or pay again for that event (A5).
BR-20	Only the event owner can approve, reject or ban. Approve and Reject apply to PENDING only. Ban applies to PENDING or APPROVED.
BR-21	Admins moderate content (delete events and users); they do not approve or reject participants of events they do not own.

8.4 Invitations

ID	Rule
BR-22	Only the event owner can invite. The invitee must be a registered user, identified by email.
BR-23	There is at most one invitation per user per event. A user who already has a participation (any status) cannot be invited.
BR-24	Free invitation: Accept creates an APPROVED participation (A1). Paid invitation: Pay & Accept, then PENDING. Decline sets DECLINED and is final.
BR-25	If a user joins through the normal join flow while holding a PENDING invitation, that invitation becomes ACCEPTED.
BR-26	An invitation for a past event cannot be accepted.

8.5 Payments

ID	Rule
BR-27	The amount is always read from the event fee on the server. The client never sends an amount.
BR-28	Currency is BDT. The amount sent to the gateway is a string with 2 decimals, for example '500.00'.
BR-29	Payment status moves PENDING to SUCCESS, FAILED or CANCELLED and then never changes.
BR-30	A payment is SUCCESS only after server-side validation with the gateway: status VALID or VALIDATED, same tranId, same amount, same currency.
BR-31	Gateway callbacks may arrive more than once or out of order. Processing must be idempotent.
BR-32	Fees are not refunded automatically in v1 (A2), including for rejected requests.
BR-33	Payment records are kept permanently, even when the user or event is deleted.

8.6 Reviews and admin

ID	Rule
BR-34	Deleting an event as an Admin or as the owner removes its participations, invitations and reviews. Payments stay.
BR-35	Only an APPROVED participant can create a review, and only after the event has started (A3).
BR-36	One review per user per event. Rating is an integer from 1 to 5. Comment is optional, up to 1000 characters.
BR-37	Only the author can edit or delete a review, and only within 7 days after it was created (constant REVIEW_EDIT_WINDOW_DAYS = 7).
BR-38	Average rating is the mean of all ratings for the event, rounded to 1 decimal. An event with no reviews shows 'No reviews yet'.

9. API Endpoints

9.1 Conventions

- Base URL: {SERVER_URL}/api/v1. JSON request and response bodies (the payment callbacks receive form-urlencoded data from the gateway).
- Auth column: Public = no token; Optional = token used if present; User = valid token; Owner = token of the event owner; Admin = role ADMIN; Gateway = called by SSLCommerz.
- Protected requests send Authorization: Bearer followed by the token. Ids are UUID strings.
- List endpoints accept page (default 1) and limit (default 12, maximum 50) and return meta (section 14).
- Route order matters in Express: fixed paths such as /events/featured, /events/upcoming and /events/mine must be declared before /events/:id.

9.2 Auth and users

Method	Path	Auth	Description
POST	/auth/register	Public	Body: name, email, password. Creates a USER. Returns token and user.
POST	/auth/login	Public	Body: email, password. Returns token and user.
GET	/auth/me	User	Returns the current user (no passwordHash).
PATCH	/users/me	User	Body: name?, phone?. Updates profile.
PATCH	/users/me/notifications	User	Body: notificationsEnabled (boolean).

9.3 Events

Method	Path	Auth	Description
GET	/events	Optional	List events. Query: search (title or organizer name), type (PUBLIC_FREE, PUBLIC_PAID, PRIVATE_FREE, PRIVATE_PAID), page, limit, includePast (default false). Upcoming first.
GET	/events/featured	Public	Featured event, or the soonest upcoming public event if none is featured.
GET	/events/upcoming	Public	Up to 9 upcoming PUBLIC events for the slider.
GET	/events/mine	User	Events created by the current user.
GET	/events/:id	Optional	Event details, organizer, averageRating, reviewCount, approvedCount and a viewer object (isOwner, participationStatus, invitationStatus). Venue and link hidden per BR-13.
POST	/events	User	Body: title, description, eventDate, venue?, eventLink?, visibility, fee.
PATCH	/events/:id	Owner	Partial update; BR-09 applies.
DELETE	/events/:id	Owner / Admin	Deletes the event (BR-34).

9.4 Participation

Method	Path	Auth	Description
POST	/events/:id/join	User	Free events only. Public + Free creates APPROVED; Private + Free creates PENDING. Paid events return 402 PAYMENT_REQUIRED.
GET	/events/:id/participants	Owner	List participants. Query: status, page, limit. Includes user name and email.
PATCH	/participations/:id/approve	Owner	PENDING to APPROVED.
PATCH	/participations/:id/reject	Owner	PENDING to REJECTED.
PATCH	/participations/:id/ban	Owner	PENDING or APPROVED to BANNED.
GET	/participations/mine	User	Events the current user has joined or requested, with status.

9.5 Invitations

Method	Path	Auth	Description
POST	/events/:id/invitations	Owner	Body: email. Creates a PENDING invitation.
GET	/events/:id/invitations	Owner	Invitations sent for this event.
GET	/invitations/mine	User	Pending invitations of the current user, with event summary.
POST	/invitations/:id/accept	User	Free events: creates APPROVED participation. Paid events: 402 PAYMENT_REQUIRED (use /payments/init).
POST	/invitations/:id/decline	User	Sets DECLINED.

9.6 Payments

Method	Path	Auth	Description
POST	/payments/init	User	Body: eventId, invitationId?. Returns tranId and gatewayUrl.
POST	/payments/success	Gateway	Browser POST from SSLCommerz. Validates, updates, then redirects the browser to CLIENT_URL/payment/success.
POST	/payments/fail	Gateway	Marks FAILED, redirects to CLIENT_URL/payment/fail.
POST	/payments/cancel	Gateway	Marks CANCELLED, redirects to CLIENT_URL/payment/cancel.
POST	/payments/ipn	Gateway	Server-to-server notification. Same verification as success. Responds 200.
GET	/payments/mine	User	Payment history of the current user.

9.7 Reviews

Method	Path	Auth	Description
GET	/events/:id/reviews	Public	Reviews with author name, plus averageRating and reviewCount.
POST	/events/:id/reviews	User	Body: rating, comment?. Rules BR-35 and BR-36.
PATCH	/reviews/:id	User	Author only, within 7 days (BR-37). Body: rating?, comment?.
DELETE	/reviews/:id	User	Author only, within 7 days.
GET	/reviews/mine	User	Reviews written by the current user.

9.8 Admin

Method	Path	Auth	Description
GET	/admin/stats	Admin	Counts: users, events, participations, successful payments.
GET	/admin/events	Admin	All events. Query: search, page, limit.
GET	/admin/users	Admin	All users. Query: search, page, limit.
DELETE	/admin/events/:id	Admin	Delete any event (BR-34).
DELETE	/admin/users/:id	Admin	Delete a user (BR-04, BR-05).
PATCH	/admin/events/:id/feature	Admin	Body: isFeatured (boolean). BR-11 applies.

9.9 Example: create event

```
POST /api/v1/events
Authorization: Bearer <token>
{
  "title": "Tech Meetup Dhaka",
  "description": "Monthly meetup for developers. Talks, networking and demos.",
  "eventDate": "2026-11-12T12:00:00.000Z",
  "venue": "Banani, Dhaka",
  "visibility": "PUBLIC",
  "fee": 0
}
```

10. Project Structure

Two repositories: planora-server and planora-client. The server is organized by feature module (routes, controller, service, validation per module).

10.1 Server (Express + Prisma)

```
planora-server/  
  prisma/  
    schema.prisma  
    migrations/  
    seed.ts          # creates the ADMIN account from env values  
  src/  
    server.ts        # starts the HTTP server  
    app.ts           # express app, middleware, routes  
    config/  
      env.ts         # validates and exports environment variables  
    lib/  
      prisma.ts      # single PrismaClient instance  
      sslcommerz.ts   # init session + validate payment helpers  
    middlewares/  
      authenticate.ts # required + optional JWT auth  
      requireRole.ts  # zod body/query/params validation  
      validate.ts  
      rateLimiter.ts  
      notFound.ts  
      errorHandler.ts # ApiError, Zod, Prisma, JWT errors  
    modules/  
      auth/  
        auth.routes.ts auth.controller.ts auth.service.ts auth.validation.ts  
      users/  
        users.*  
      events/  
        events.*  
      participations/  
        participations.*  
      invitations/  
        invitations.*  
      payments/  
        payments.*  
      reviews/  
        reviews.*  
      admin/  
        admin.*  
    utils/  
      ApiError.ts  
      sendResponse.ts  
      catchAsync.ts  
      jwt.ts  
      constants.ts   # REVIEW_EDIT_WINDOW_DAYS = 7, pagination defaults  
  .env.example  
  package.json  
  tsconfig.json  
  README.md
```

10.2 Client (Next.js App Router)

```
planora-client/  
src/  
  app/  
    layout.tsx  
    page.tsx # Home  
    events/page.tsx # Events list  
    events/[id]/page.tsx # Event details  
    login/page.tsx  
    register/page.tsx  
    about/page.tsx contact/page.tsx privacy-policy/page.tsx  
    payment/success/page.tsx payment/fail/page.tsx payment/cancel/page.tsx  
    dashboard/  
      layout.tsx # sidebar layout  
      page.tsx # redirects to my-events  
      my-events/page.tsx  
      my-events/new/page.tsx  
      my-events/[id]/edit/page.tsx  
      my-events/[id]/participants/page.tsx  
      invitations/page.tsx  
      reviews/page.tsx  
      settings/page.tsx  
      admin/page.tsx admin/events/page.tsx admin/users/page.tsx  
  components/  
    ui/ Button, Input, Select, Modal, Badge, Spinner, Pagination ...  
    layout/ Navbar, Footer, DashboardSidebar  
    home/ Hero, UpcomingSlider, CategoryFilters, CallToAction  
    events/ EventCard, EventForm, EventActions, ParticipantTable, ReviewList, ReviewForm  
  context/ AuthContext.tsx  
  hooks/ useAuth.ts, useDebounce.ts  
  lib/  
    api.ts # axios instance + interceptors  
    auth.ts # cookie helpers  
    validations.ts # zod schemas  
    format.ts # date and money formatting  
  types/ index.ts  
  middleware.ts # redirects guests away from /dashboard/*  
  .env.example  
  tailwind.config.ts  
  package.json  
  README.md
```

11. Technical Specifications

Area	Specification
Runtime and language	Node.js 20 LTS or newer, TypeScript on both client and server.
Frontend	Next.js (App Router), React, Tailwind CSS. Forms with React Hook Form + Zod. Slider with Swiper.
Backend	Express.js REST API under /api/v1, feature-module structure, central error handler.
Database and ORM	PostgreSQL 15 or newer. Prisma ORM with migrations (prisma migrate dev locally, prisma migrate deploy in production).
Authentication	JWT (HS256), bcrypt password hashing. Section 3.
Payments	SSLCommerz hosted checkout. Sandbox for development and testing, live credentials for production. Currency BDT.
Validation	Zod schemas on the server for body, query and params; matching Zod schemas on the client forms.
Security	helmet, CORS restricted to CLIENT_URL, rate limiting (general API and a stricter limit on /auth/login and /auth/register), parameterized queries through Prisma, secrets only in environment variables, HTTPS in production, no stack traces in production responses.
Pagination	page and limit query params; default limit 12, maximum 50.
Dates and time zone	Stored and transmitted in UTC (ISO 8601). Displayed in the viewer's local time zone.
Money	Decimal(10,2) in the database; formatted as BDT in the UI; sent to the gateway as a 2-decimal string.
Logging	morgan request logging; errors logged server-side with the request id.
Performance targets	List and detail endpoints respond in under 500 ms at normal load on a hosted free tier (target, not a guarantee). Indexes as in section 7.1.
Browsers	Latest two versions of Chrome, Edge, Firefox and Safari; Android and iOS mobile browsers.
Hosting (suggested)	Client on Vercel; server on Render, Railway or similar; PostgreSQL on Neon, Supabase, Railway or similar. Any equivalent host is fine.

11.1 Environment variables

Variable	Where	Purpose
DATABASE_URL	Server	PostgreSQL connection string.
PORT	Server	HTTP port (default 5000).
NODE_ENV	Server	development or production.
JWT_SECRET	Server	Signing secret, at least 32 random characters.
JWT_EXPIRES_IN	Server	Token lifetime, 7d.
CLIENT_URL	Server	Allowed CORS origin and redirect target after payment.
SERVER_URL	Server	Public server URL used to build gateway callback URLs.
SSLCZ_STORE_ID	Server	SSLCommerz store id.
SSLCZ_STORE_PASSWORD	Server	SSLCommerz store password. Secret.
SSLCZ_IS_LIVE	Server	false = sandbox, true = live.
ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD	Server	Used only by the seed script to create the Admin.
NEXT_PUBLIC_API_URL	Client	Base URL of the API, for example https://api.example.com/api/v1.

SSLCommerz field names and endpoint URLs can change. Before coding the integration, check the current SSLCommerz developer documentation for the session-creation and validation APIs and confirm the exact required fields.

12. Dependencies

Install the latest stable release of each package and commit the lockfile so everyone builds the same versions.

12.1 Server runtime

Package	Purpose
express	HTTP server and routing.
@prisma/client	Database client generated from schema.prisma.
jsonwebtoken	Sign and verify JWTs.
bcryptjs	Password hashing (pure JavaScript, no native build).
zod	Request validation and environment validation.
cors	CORS policy.
helmet	Security headers.
express-rate-limit	Rate limiting.
morgan	HTTP request logging.
dotenv	Load .env in development.
axios	Call the SSLCommerz session and validation APIs.

12.2 Server development

Package	Purpose
prisma	CLI: migrate, generate, studio, seed.
typescript, tsx	Compile and run TypeScript with watch mode.
@types/node, @types/express, @types/jsonwebtoken, @types/cors, @types/morgan	Type definitions.
eslint, prettier	Linting and formatting.

12.3 Client runtime

Package	Purpose
next, react, react-dom	Framework and UI library.
tailwindcss (with postcss, autoprefixer as the installed version requires)	Styling.
axios	API client with interceptors.
react-hook-form, zod, @hookform/resolvers	Forms and validation.
swiper	Upcoming events slider.
react-hot-toast	Toast notifications.
lucide-react	Icons.
js-cookie	Read and write the planora_token cookie.
date-fns	Date formatting.

12.4 Client development

Package	Purpose
typescript, @types/react, @types/node, @types/js-cookie	Types.
eslint, eslint-config-next, prettier	Linting and formatting.

13. Business Logic Rules

These are the step-by-step procedures the service layer must follow. Every database change that spans more than one table runs inside a single Prisma transaction.

13.1 BL-01 Join eligibility (used by join, payment init and invitation accept)

#	Check	If it fails
1	Request has a valid token.	401 UNAUTHORIZED
2	Event exists.	404 NOT_FOUND
3	event.eventDate is in the future.	400 BAD_REQUEST: 'This event has already started or ended.'
4	Current user is not event.organizerId.	403 FORBIDDEN: 'You cannot join your own event.'
5	No participation exists for (event, user), or the existing one is not PENDING, APPROVED, REJECTED or BANNED.	PENDING or APPROVED: 409 CONFLICT 'Already joined or requested.' REJECTED: 403 'Your request was rejected.' BANNED: 403 'You are banned from this event.'

13.2 BL-02 Join on a free event

- Run BL-01.
- If event.fee is above 0: return 402 PAYMENT_REQUIRED ('Payment required. Use Pay & Join.').
- If visibility = PUBLIC: create Participation with status APPROVED.
- If visibility = PRIVATE: create Participation with status PENDING.
- In the same transaction, set any PENDING invitation for (event, user) to ACCEPTED (BR-25).
- A unique-constraint error (Prisma P2002) from a double click is mapped to 409 CONFLICT.

13.3 BL-03 Payment initiation (POST /payments/init)

- Run BL-01. If invitationId is given, it must belong to the current user, match the event, and be PENDING; otherwise 400.
- If event.fee is 0: return 400 'This event is free.'
- Generate tranId = 'PLN-' + random unique string. Create Payment { status PENDING, amount = event.fee, currency BDT, eventTitle = event.title, invitationId }.
- Call the SSLCommerz session API with: store credentials, total_amount (2-decimal string), currency BDT, tran_id, success_url, fail_url, cancel_url, ipn_url (all under SERVER_URL/api/v1/payments/...), customer name, email and phone, product name = event title, and the other customer and product fields the gateway requires (use placeholders such as 'N/A', 'Dhaka', 'Bangladesh' when the user has none).
- If the gateway does not answer with status SUCCESS: set Payment FAILED and return 502 GATEWAY_ERROR.
- Otherwise return { tranId, gatewayUrl } where gatewayUrl is the GatewayPageURL from the gateway.

13.4 BL-04 Payment verification (success callback and IPN)

- Read tran_id, val_id and status from the form body. Find the Payment by tranId. If not found: respond 404 for IPN, or redirect to the fail page for the browser.
- If Payment.status is not PENDING: do nothing more (idempotent) and redirect according to the stored status.
- If the gateway status is not VALID or VALIDATED: set Payment FAILED and redirect to the fail page.
- Call the gateway validation API with val_id and the store credentials. Require status VALID or VALIDATED, tran_id equal to Payment.tranId, currency equal to BDT, and amount equal to Payment.amount (compare as decimals). Any mismatch: set Payment FAILED and log it.
- Start a transaction. Update the Payment only if it is still PENDING (updateMany with a status condition) to SUCCESS with valId, bankTranId and paidAt. If nothing was updated, another callback already handled it: stop.
- If no Participation exists for (event, user): create one with status PENDING and link it to the Payment. If invitationId is set, mark the invitation ACCEPTED with respondedAt.

- If a Participation already exists (a duplicate payment), keep it unchanged, keep the Payment as SUCCESS and write a warning log so the amount can be refunded manually.
- Redirect the browser to CLIENT_URL/payment/success?tran_id=... For IPN respond 200 with a short body.
- Fail and cancel endpoints: set Payment to FAILED or CANCELLED only if it is still PENDING, then redirect to the matching client page.

13.5 BL-05 Invitation accept and decline

- Invitation must belong to the current user and be PENDING; the event must be in the future.
- Accept on a free event: transaction that sets the invitation ACCEPTED (respondedAt = now) and creates Participation APPROVED (A1). If a participation already exists, return 409.
- Accept on a paid event: return 402 PAYMENT_REQUIRED; the client then starts BL-03 with invitationId.
- Decline: set DECLINED with respondedAt = now. No participation is created.

13.6 BL-06 Moderation transitions

Action	Allowed from	Result	Who
approve	PENDING	APPROVED	Event owner
reject	PENDING	REJECTED	Event owner
ban	PENDING, APPROVED	BANNED	Event owner

Steps: load the participation with its event; if the current user is not event.organizerId return 403; if the current status is not in 'Allowed from' return 409 CONFLICT 'Invalid status change'; update status and set decidedAt = now.

13.7 BL-07 Reviews

- Create: user has an APPROVED participation, event.eventDate is not in the future, and no review exists yet; otherwise 403 or 409 with a clear message.
- Edit and delete: the review's userId equals the current user (else 403) and now minus createdAt is at most 7 days (else 403 'The review period has ended.').
- Event details and review list return averageRating (mean of ratings rounded to 1 decimal, or null) and reviewCount computed with a Prisma aggregate.

13.8 BL-08 Featured event

- Set: event must exist and be PUBLIC. In one transaction set isFeatured = false on all events where it is true, then set it true on the chosen event.
- Clear: set isFeatured = false.
- GET /events/featured: return the featured event; if none, the PUBLIC event with the nearest future eventDate; if none, return null and the hero shows an empty state.

13.9 BL-09 Search and filter mapping

Query	Prisma where clause
type = PUBLIC_FREE	visibility = PUBLIC and fee = 0
type = PUBLIC_PAID	visibility = PUBLIC and fee greater than 0
type = PRIVATE_FREE	visibility = PRIVATE and fee = 0
type = PRIVATE_PAID	visibility = PRIVATE and fee greater than 0
search = text	title contains text (case-insensitive) OR organizer.name contains text (case-insensitive)
default	eventDate greater than now, ordered by eventDate ascending; includePast = true removes the date condition

13.10 BL-10 Viewer context and field hiding on event details

- If a token is present, compute viewer.isOwner, viewer.participationStatus (or null) and viewer.invitationStatus (or null).

- For PRIVATE events: include venue and eventLink only if viewer is the owner, an Admin, or has participationStatus APPROVED. Otherwise return them as null and let the client show 'Visible after approval'.
- The client chooses the action button from visibility, fee and viewer state: not logged in shows 'Login to join'; owner shows owner controls; PENDING shows 'Request pending'; APPROVED shows 'Joined'; REJECTED or BANNED shows a disabled state with the reason.

13.11 BL-11 Event update and delete

- Update: owner only. If the event has any participation, invitation or payment and the body changes visibility or fee, return 409 CONFLICT (BR-09). eventDate must stay in the future.
- Delete: owner or Admin. Prisma cascades remove participations, invitations and reviews; Payment foreign keys are set to null so records stay.

13.12 BL-12 Error handling

- Services throw ApiError(status, errorCode, message). The central handler converts: ZodError to 400 VALIDATION_ERROR with field list; JsonWebTokenError to 401 UNAUTHORIZED; TokenExpiredError to 401 TOKEN_EXPIRED; Prisma P2002 to 409 CONFLICT; Prisma P2025 to 404 NOT_FOUND; anything else to 500 INTERNAL_ERROR with a generic message.
- Unknown routes return 404 NOT_FOUND through the notFound middleware.

14. Response Format

14.1 Success

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "<jwt>",
    "user": { "id": "7clf...", "name": "Rahim Uddin", "email": "rahim@example.com", "role": "USER" }
  }
}
```

14.2 Paginated list

```
{
  "success": true,
  "message": "Events fetched",
  "data": [
    {
      "id": "alb2...", "title": "Tech Meetup Dhaka", "eventDate": "2026-11-12T12:00:00.000Z",
      "visibility": "PUBLIC", "fee": "0.00", "organizer": { "id": "7clf...", "name": "Rahim Uddin" }
    }
  ],
  "meta": { "page": 1, "limit": 12, "total": 37, "totalPages": 4 }
}
```

14.3 Event details with viewer context

```
{
  "success": true,
  "message": "Event fetched",
  "data": {
    "id": "alb2...", "title": "Design Workshop", "visibility": "PRIVATE", "fee": "500.00",
    "eventDate": "2026-11-20T09:00:00.000Z", "venue": null, "eventLink": null,
    "organizer": { "id": "7clf...", "name": "Rahim Uddin" },
    "averageRating": 4.5, "reviewCount": 8, "approvedCount": 8,
    "viewer": { "isOwner": false, "participationStatus": null, "invitationStatus": null }
  }
}
```

14.4 Payment initiation

```
{
  "success": true,
  "message": "Payment session created",
  "data": { "tranId": "PLN-8F3K2Q9X", "gatewayUrl": "https://sandbox.sslcommerz.com/..." }
}
```

14.5 Error

```
{
  "success": false,
  "message": "Validation failed",
  "errorCode": "VALIDATION_ERROR",
  "errors": [
    { "field": "fee", "message": "Fee must be 0 or more" },
    { "field": "venue", "message": "Provide a venue or an event link" }
  ]
}
```

The errors array is present only for validation errors. The client shows message in a toast and maps errors[].field to the matching form field.

14.6 Status codes and error codes

HTTP	errorCode	When it is used
200 / 201	-	Success / resource created.
400	VALIDATION_ERROR, BAD_REQUEST	Invalid input or a rule such as 'event has already started'.
401	UNAUTHORIZED, TOKEN_EXPIRED	Missing, invalid or expired token; wrong credentials.
402	PAYMENT_REQUIRED	Free-join or accept called for a paid event.
403	FORBIDDEN	Not allowed (role, not the owner, banned, rejected, review period over).
404	NOT_FOUND	Resource or route does not exist.
409	CONFLICT	Duplicate email, duplicate participation, invalid status change, locked fields.
429	RATE_LIMITED	Too many requests.
500	INTERNAL_ERROR	Unexpected server error (generic message).
502	GATEWAY_ERROR	Payment gateway did not create a session.

15. Deployment Checklist

Tick each item before submission. [] = to do.

15.1 Before deploying

Done	Item
[]	Both repositories on GitHub with at least 20 meaningful commits each (feature-based messages, no 'update' or 'fix' spam).
[]	README in each repository: purpose, features, tech stack, setup steps, environment variables, scripts, live links.
[]	No secrets committed. .env is in .gitignore and .env.example lists every variable.
[]	Server and client build locally without errors (npm run build).
[]	All four event types tested end to end locally with the SSLCommerz sandbox.

15.2 Database

Done	Item
[]	Production PostgreSQL created; DATABASE_URL set on the server host.
[]	npx prisma migrate deploy run against production.
[]	Seed script run once to create the Admin account; Admin credentials noted for the submission.
[]	Indexes from section 7.1 exist (they come from the migrations).

15.3 Server

Done	Item
[]	Environment variables from section 11.1 set; JWT_SECRET is a new random value (not the development one).
[]	NODE_ENV=production; CORS origin equals the exact client URL (https, no trailing slash).
[]	Health endpoint (for example GET /health) returns 200.
[]	express.urlencoded is enabled so the gateway callbacks can be parsed.
[]	Rate limiting on and error responses contain no stack traces.

15.4 Client

Done	Item
[]	NEXT_PUBLIC_API_URL points to the production API.
[]	Cookie is Secure on HTTPS; /dashboard redirects guests to /login.
[]	Home, Events, Event Details, Login, Signup, Dashboard, Admin and payment result pages load without console errors.
[]	Layout checked on mobile, tablet and desktop widths.

15.5 Payments

Done	Item
[]	SERVER_URL is the public HTTPS URL; success, fail, cancel and IPN URLs are reachable from the internet.
[]	Sandbox flow verified: success, failure and cancel each end on the correct client page with the correct database state.
[]	Switch SSLCOMMERZ to live credentials only if real payments are required (SSLCZ_IS_LIVE=true); otherwise keep sandbox and say so in the README.
[]	Duplicate callback test: replaying the success callback does not create a second participation.

15.6 Post-deploy smoke test

Done	Item
[]	Register a new user and log in.
[]	Create a Public Free, Public Paid, Private Free and Private Paid event.
[]	Second user joins the Public Free event (APPROVED instantly).
[]	Second user pays for the Public Paid event; participation shows PENDING; host approves.
[]	Second user requests the Private Free event; host rejects one request and bans another participant.
[]	Host sends an invitation; invitee accepts (free) and pays and accepts (paid).
[]	Participant writes, edits and deletes a review within the window.
[]	Admin logs in, deletes an event and a user, sets the featured event; hero updates.

15.7 Submission

Done	Item
[]	Video 5 to 10 minutes covering: registration, login, create event, public free join, paid event payment, private join request, host approval, dashboard features, admin moderation, reviews.
[]	Live client URL, live server URL, both GitHub links and Admin test credentials included in the submission.

16. Future Considerations (Post-v1)

Area	Idea	Value
Payments	Automated refunds when a host rejects or an event is deleted; organizer payouts; add ShurjoPay as a second gateway.	Closes the biggest trust gap (A2).
Notifications	Real email and in-app notifications for approvals, invitations, payment results and reminders, driven by the existing Notifications setting.	Keeps users informed without checking the dashboard.
Security	Refresh tokens with httpOnly cookies, email verification, password reset, optional two-factor login, token revocation list.	Stronger session security.
Capacity	Maximum attendees, waitlist, auto-approve option for hosts.	Fits real venues.
Moderation	Unban and re-approve actions, report event or review, Admin review deletion, audit log, soft delete for events and users.	Safer and reversible moderation.
Events	Banner image upload, categories and tags, recurring events, co-hosts, calendar export (ICS).	Richer discovery.
Tickets	QR ticket after approval and check-in screen for hosts.	On-site entry control.
Discovery	Full-text search, location filter and map, recommendations.	Better results at scale.
Experience	Bangla language support, dark mode, real-time updates with WebSockets.	Wider audience.
Engineering	Automated tests (unit, API, end to end), CI/CD pipeline, Docker, monitoring and error tracking, background job queue for callbacks.	Reliability and faster releases.
Analytics	Host dashboard with registrations, revenue and rating trends; Admin analytics.	Insight for hosts and the platform.

End of document